



Современные архитектуры ИС



Лекция 6
Улучшение архитектуры трансформера



План лекции

1. Нормализации
2. Gated FFN
3. Улучшение позиционного кодирования
4. Улучшение остаточных связей
5. KV-кэш
6. Разреженное внимание

Пост-нормализация

В оригинальной архитектуре трансформера использовалась **пост-нормализация**

$$y = \text{LayerNorm}(x + f(x))$$

Но если представить остаточные связи как магистраль для распространения градиентов, то такие нормы будут создавать блоки:

$$\frac{\partial y}{\partial x} = \underbrace{\frac{\partial \text{LN}(\tilde{z})}{\partial \tilde{z}} \frac{\partial \text{LN}(\tilde{x})}{\partial (\tilde{x})}}_{\text{Градиенты почти не затухают}} \cdot \left(1 + \underbrace{\frac{\partial \text{FFN}(z)}{\partial z} + \frac{\partial \text{MHA}(x)}{\partial x}}_{\text{Градиенты затухают слабее}} + \underbrace{\frac{\partial \text{FFN}(z)}{\partial z} \cdot \frac{\partial \text{MHA}(x)}{\partial x}}_{\text{Градиенты сильно затухают}} \right)$$

Пре-нормализация

Чтобы избежать проблемы распространения градиентов, поменяем порядок операций:

$$y = x + f(\text{LayerNorm}(x))$$

Но такая нормализация может привести к численной неустойчивости обучения

Сэндвич-нормализация

Чтобы улучшить стабильность обучения, мы можем использовать сэндвич-нормализацию:

$$y = x + \text{LayerNorm} \left[f(\text{LayerNorm}(x)) \right]$$

Но и у этого подхода есть издержки

RMSNorm

Мы хотим меньше считывать данных из глобальной памяти и мы хотим считать меньше редукций:

$$\text{RMSNorm}(x) = \gamma \frac{x}{\sqrt{\frac{1}{d} \sum_{i=1}^d x_i^2 + \epsilon}}$$

План лекции

1. Нормализации
2. **Gated FFN**
3. Улучшение позиционного кодирования
4. Улучшение остаточных связей
5. KV-кэш
6. Разреженное внимание

Шлюз активаций

Идея многих функций активации заключается в создании некоторого шлюза активаций. “Хорошие” активации мы пропускаем вперед, “плохие” — зануляем.

Но делаем мы это по заранее заготовленному правилу. А что, если мы позволим модели самой выбирать оптимальный шлюз?

$$\text{GLU}(x) = f(xW_1) \otimes (xW_2),$$

SwiGLU

Одна из наиболее популярных GLU-функций активации — SwiGLU, построенная на основе SiLU функции активации:

$$\text{SiLU}_{\beta=1}(x) = x\sigma(x)$$

$$\text{SwiGLU}(x) = \text{SiLU}(xW_1) \otimes (xW_2)$$

План лекции

1. Нормализации
2. Gated FFN
3. **Улучшение позиционного кодирования**
4. Улучшение остаточных связей
5. KV-кэш
6. Разреженное внимание

Не обучаемое и обучаемое позиционное кодирование

Синусоидальное не обучаемое позиционное кодирование (из оригинального трансформера):

$$\begin{aligned}\text{PE}(\text{pos}, 2i) &= \sin\left(\frac{\text{pos}}{10000^{2i/D}}\right), \\ \text{PE}(\text{pos}, 2i + 1) &= \cos\left(\frac{\text{pos}}{10000^{2i/D}}\right),\end{aligned}$$

Обучаемое позиционное кодирование:

$$\hat{x}_{\text{pos}} = x_{\text{pos}} + \text{PE}(\text{pos})$$

Относительное позиционное кодирование

Абсолютное позиционное кодирование, особенно обучаемое, тяжело масштабировать на длинные контексты. Поэтому рассмотрим относительное позиционное кодирование:

Обычное внимание

$$\text{score}(i, j) = \frac{Q_i K_j^T}{\sqrt{d}} = \frac{x_i W_Q (x_j W_K)^T}{\sqrt{d}}$$

Внимание с относительным
позиционным кодированием

$$\text{score}(i, j) = \frac{x_i W_Q (x_j W_K + a_{ij})^T}{\sqrt{d}},$$

RoPE

Совместим идеи относительного и абсолютного позиционных кодирований: будем вращать вектора

$$R(q, p) = \begin{pmatrix} q_1 \\ q_2 \\ \dots \\ q_d \end{pmatrix}^\top \begin{bmatrix} M_1 & 0 & \dots & 0 \\ 0 & M_2 & \dots & 0 \\ \dots & \dots & \dots & \dots \\ 0 & \dots & 0 & M_{d/2} \end{bmatrix}$$

$$M_k = \begin{pmatrix} \cos(p\omega_k) & \sin(p\omega_k) \\ -\sin(p\omega_k) & \cos(p\omega_k) \end{pmatrix}$$

RoPE

Частоты – такие же как и в синусоидальном позиционном кодировании

$$\omega_k = \frac{1}{10000^{2(k-1)/d}}$$

Механизм внимания принимает вид:

$$\text{score}(i, j) = \frac{R(x_i W_Q, i) R(x_j W_K, j)^T}{\sqrt{d}}$$

План лекции

1. Нормализации
2. Gated FFN
3. Улучшение позиционного кодирования
4. **Улучшение остаточных связей**
5. KV-кэш
6. Разреженное внимание

Гиперсвязи

В классическом residual connection всего одна связь. Увеличим выразительную силу, храня больше связей. Но обрабатывать каждую связь — дорого. Поэтому вычислим среднюю взвешенную сумму

$$\hat{x} = \sum_{i=1}^R h_i^{\text{pre}} x_i,$$

$$\hat{z} = f(\hat{x}),$$

А после разагрегируем:

$$z_i = h^{\text{post}} \hat{z}$$

$$y_i = x_i + z_i$$

Гиперсвязи

Чтобы увеличить обмен информацией, перемешаем активации в гиперсвязях:

$$y_i = \sum_{j=1}^R h_j^{\text{res}} x_j + z_i,$$

Запишем итог в матричной форме (в данном случае — в col-major форме):

$$y = H^{\text{res}} x + H^{\text{post}} f \left[H^{\text{pre}} x \right]$$

Динамические гиперсвязи

Сделаем гиперсвязи адаптивными к гипервходам:

$$H^{\text{pre}}(x) = \alpha^{\text{pre}} \tanh(\theta^{\text{pre}} x^\top) + b^{\text{pre}},$$

$$H^{\text{post}}(x) = \alpha^{\text{post}} \tanh(\theta^{\text{post}} x^\top) + b^{\text{post}},$$

$$H^{\text{res}}(x) = \alpha^{\text{res}} \tanh(\theta^{\text{res}} x^\top) + b^{\text{res}}$$

Но такой подход приводит к неустойчивости в обучении

Нестабильность динамических гиперсвязей

Запишем выражение для динамических связей

$$x_{l+1} = H_l^{\text{res}}(x_l) + H_l^{\text{post}} \left(f_l \left[H_l^{\text{res}}(x_l) \right] \right)$$

Упростим запись

$$x_{l+1} = H_l^{\text{res}} x_l + H_l^{\text{post}} f_l \left[H_l^{\text{res}} x_l \right]$$

Нестабильность динамических гиперсвязей

Выполним преобразования дальше, вплоть до финального слоя L:

$$x_L = \left(\prod_{i=1}^{L-l} H_{L-i}^{\text{res}} \right) x_l + \sum_{i=l}^{L-1} \left(\prod_{j=1}^{L-1-i} H_{L-j}^{\text{res}} \right) H_i^{\text{post}} f \left(H_i^{\text{pre}} x_i \right)$$

Сравним с обычными остаточными связями:

$$x_L = x_l + \sum_{i=l}^L f_i(x_i)$$

Дважды стохастическая матрица

Дважды стохастическая матрица — это матрица, все элементы которой положительны, а сумма элементов каждой строки и каждого столбца равна 1.

Алгоритм Синкхорна-Кноппа

Чтобы получить дважды стохастическую матрицу, будем итеративно масштабировать столбцы и строки к сумме элементов 1.

Сначала отмасштабируем все столбцы к единичной сумме. Потом — строки. После этого повторяем. Это позволяет за ограниченное число итераций приблизить матрицу к дважды стохастической.

mHC

Построим ограниченные в многообразии гиперсвязи (Manifold-Constrained Hyper-Connections)

$$\begin{cases} H^{\text{pre}} = \sigma\left(\alpha_l^{\text{pre}}(x_l \varphi_l^{\text{pre}})\right) + b_l^{\text{pre}}, \\ H^{\text{post}} = 2\sigma\left(\alpha_l^{\text{post}}(x_l \varphi_l^{\text{post}})\right) + b_l^{\text{post}}, \\ H_l^{\text{res}} = \text{Sinkhorn-Knopp}\left[\exp\left(\alpha_l^{\text{res}} \text{mat}(x_l \varphi_l^{\text{res}}) + b_l^{\text{res}}\right)\right] \end{cases}$$

План лекции

1. Нормализации
2. Gated FFN
3. Улучшение позиционного кодирования
4. Улучшение остаточных связей
5. **KV-кэш**
6. Разреженное внимание

Идея KV-кэш

При обучении модель предсказывает следующий токен для каждого токена последовательности. В этой задаче очевидно, что необходимо вычислять матрицу внимания целиком.

Однако, при инференсе, нам важно предсказать лишь один следующий токен.

Идея KV-кэш

На каждом новом шаге генерации мы используем информацию с предыдущих шагов

$$Q_{t+1} = X_{t+1}W_Q = \begin{bmatrix} X_t \\ x_{t+1} \end{bmatrix} W_Q = \begin{bmatrix} Q_t \\ x_{t+1}W_Q \end{bmatrix},$$
$$K_{t+1} = \begin{bmatrix} K_t \\ x_{t+1}W_K \end{bmatrix}, \quad V_{t+1} = \begin{bmatrix} V_t \\ x_{t+1}W_V \end{bmatrix}$$

В таком случае, зачем считать матрицу Q целиком, когда нам нужен лишь один ее вектор? И зачем нам заново считать K и V, если мы можем взять из с предыдущих шагов?

Проблемы KV-кэша

KV-кэш позволяет решить проблему квадратичной сложности вычисления внимания, НО, потребляет очень много памяти:

$$\text{size(KV-cache)} = \text{batch_size} \times \text{seq_length} \times N_l \times N_h \times 2 \times d_{\text{head}} \times \text{dtype_size}$$

MQA

Один из главных “потребителей” памяти в KV-кэше — головы внимания.

Избавимся от хранения KV-кэша для разных голов, будем использовать лишь одну пару матриц K и V на каждом слое.

$$Q_i = XW_{Q,i}, \quad K = XW_K, \quad V = XW_V$$

Но таким образом мы сильно сокращаем выразительную силу модели

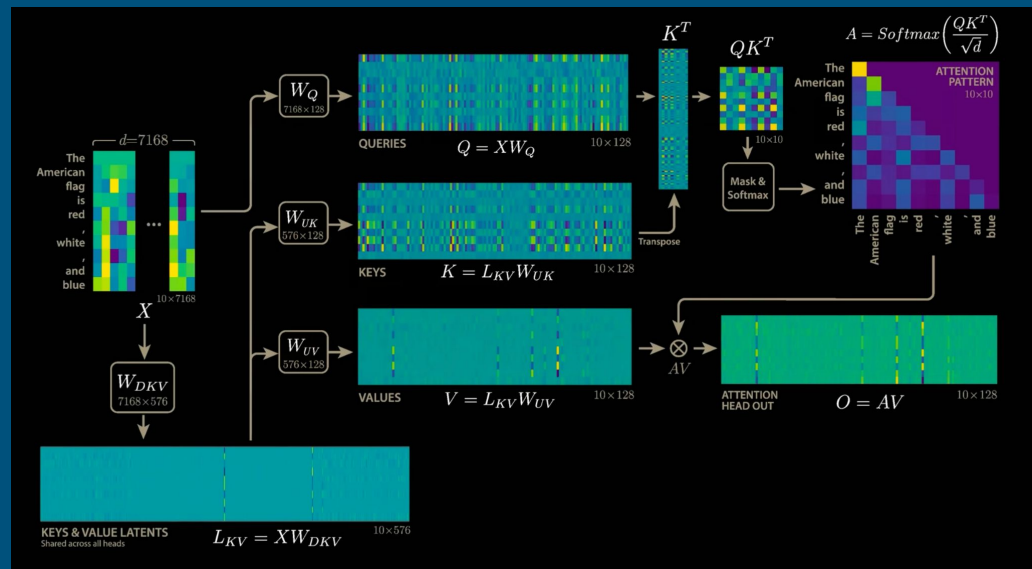
GQA

Компромиссный вариант между МНА и MQA — GQA. Давайте разобьем матрицы Q на группы. У каждой такой группы будут свои матрицы K и V .

Если группа всего одна — получаем MQA. Если количество групп равно количеству голов — получаем МНА.

Многоглавое скрытое внимание (MLA)

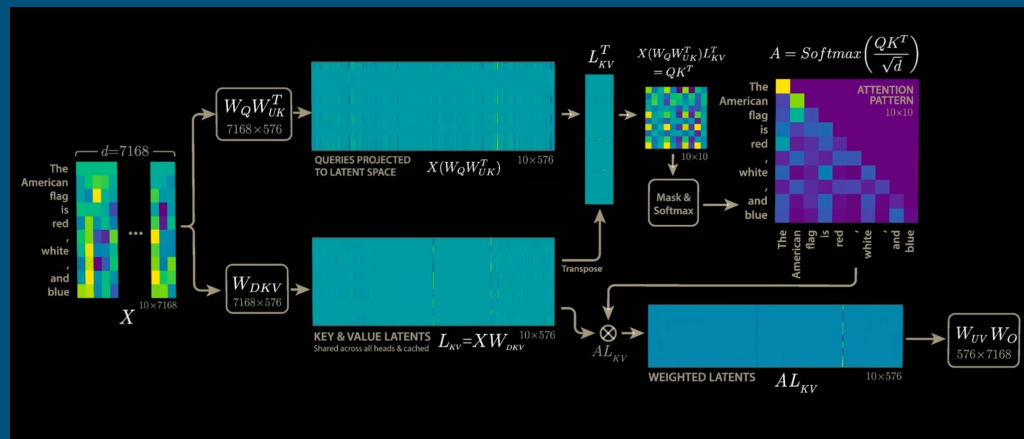
А что если KV-кэш мы будем хранить в сжатом виде для всех голов сразу?



Линейная алгебра MLA

Наивный MLA добавляет дополнительные матричные умножения, которые усложняют механизм внимания.

Упростим его



Линейная алгебра MLA

$$L_{KV} = XW_L, \quad K = L_{KV}W_K, \quad V = L_{KV}W_V$$

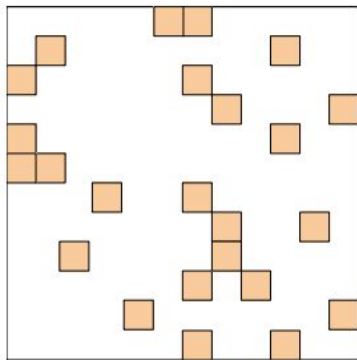
$$S = QK^\top = XW_Q(L_{KV}W_K)^\top = X \underbrace{W_Q W_K^\top}_{\widetilde{W}_Q} L_{KV}^\top = X \widetilde{W}_Q L_{KV}^\top,$$

$$\begin{aligned} O &= \begin{bmatrix} O_{h_1} \\ O_{h_2} \\ \dots \\ O_{h_n} \end{bmatrix} W_O = \begin{bmatrix} A_1 V_1 \\ A_2 V_2 \\ \dots \\ A_n V_n \end{bmatrix} W_O = \begin{bmatrix} A_1 L_{KV} W_{V,1} \\ A_2 L_{KV} W_{V,2} \\ \dots \\ A_n L_{KV} W_{V,n} \end{bmatrix} W_O = \\ &= \begin{bmatrix} A_1 L_{KV} \\ A_2 L_{KV} \\ \dots \\ A_n L_{KV} \end{bmatrix} \underbrace{\text{diag}(\{W_{V,i}\}_{i=1}^n)}_{\widetilde{W}_O} W_O = \begin{bmatrix} A_1 L_{KV} \\ A_2 L_{KV} \\ \dots \\ A_n L_{KV} \end{bmatrix} \widetilde{W}_O \end{aligned}$$

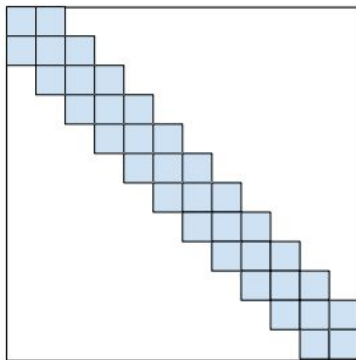
План лекции

1. Нормализации
2. Gated FFN
3. Улучшение позиционного кодирования
4. Улучшение остаточных связей
5. KV-кэш
6. Разреженное внимание

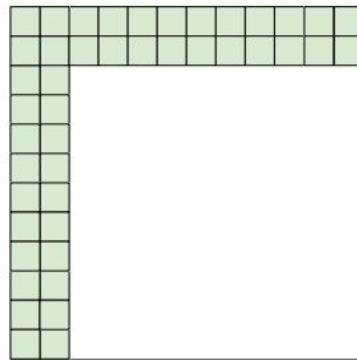
Шаблонное разреженное внимание



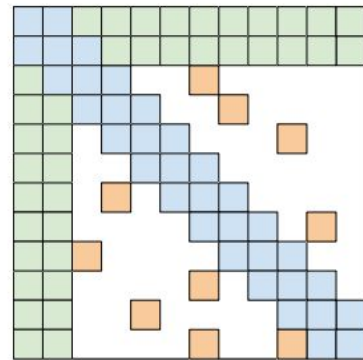
(a) Random attention



(b) Window attention



(c) Global Attention



(d) BIGBIRD

Вопросы?
